

Vyomi — Fidelity Master Plan (AWS / GCP / Azure)

Status: living document. Owner: platform. Created 2026-05-25.

1. North Star

Simulate the main cloud providers (AWS, GCP, Azure) fully on localhost so that a **real, unmodified application** — built with real SDKs, CLIs, and Terraform — can be **deployed, tested, and validated** against the simulator, then **exported to the real cloud via Terraform seamlessly** (repoint the endpoint, `terraform apply` works).

This is a *simulator*, not a reimplementation of cloud internals. We are faithful only at the application-facing boundary; behind it we use the cheapest convincing open-source tech.

2. Governing Principles

1. **The faithfulness boundary is the application interface.** Apps are never programmed to fit the simulator. If a real SDK/CLI/Terraform client can't tell the difference on the wire, we've succeeded — regardless of what runs behind it.
2. **Simulate the contract + lifecycle; fake the substrate.** Never reproduce hypervisors, distributed storage, replication, placement, or networking internals.
3. **Reuse aggressively.** Drive payload exactness from the providers' own machine-readable models; back data-plane services with protocol-compatible OSS.
4. **Conformance is the only definition of "done."** Real clients are the judge. "Integrated" means a measured parity %, not "a handler exists."
5. **Incremental correction, not rewrite.** The existing code already clears part of the bar (real `aws-cli` runs the S3 lifecycle today). Correct it service-by-service, conformance-gated.

3. The Do-Not-Deviate Surface (the spec)

Must match the real provider:

- Endpoints, paths, protocols (REST-XML / AWS query / JSON / rest-json; GCP REST/Discovery).
- Request + response **payloads**, headers, status codes, and **error shapes**.
- **Auth flow**: accept real SDK-signed requests (SigV4, GCP OAuth/ADC). Dummy creds OK; the signature must not be *rejected*. (We do not verify identity — see §5.)
- **SDK mechanics**: pagination tokens, retries, **waiters**, **long-running Operations**.
- **Lifecycle semantics**: states, timing, read-after-write, idempotency; ID/ARN/self-link formats.
- **Terraform expectations**: stable IDs, read-after-write, **clean plan after apply** (no perpetual diff).

Free to fake: storage/compute/network substrate, durability, performance, multi-AZ, scaling, real auth verification, billing.

4. Control Plane vs Data Plane (the backing-tech rule)

Pick the lightest OSS that speaks the same protocol the application speaks.

Service (AWS / GCP / Azure)	Plane	Backing tech	Notes
S3 / Cloud Storage / Blob	data	MinIO or S3-compatible store	app does real GET/PUT/multipart; move bytes out of state blob
EC2 / Compute Engine / VMs	data	LXD/Multipass (already built)	real SSH/console; timed state machine on top
RDS / Cloud SQL / Azure SQL	data	real Postgres/MySQL in a container	app runs real SQL; not SQLite
Lambda / Functions / Functions	data	subprocess/container exec	actually execute user code; enforce timeout
DynamoDB / Firestore / Cosmos	data	SQLite (FirestoreEngine pattern exists)	KV/doc semantics, TTL reaper
SQS / Pub/Sub / Service Bus	data	queue + timer (in-proc/SQLite)	real delivery/visibility/retention/DLQ
IAM / IAM / Entra	control	existing policy evaluator	shape + lifecycle; identity not verified
VPC / VPC / VNet, API GW, DNS	control	logical model (mostly exists)	shape + lifecycle

Control-plane fidelity = API shape + lifecycle (light backing fine). Data-plane fidelity = protocol-compatible OSS because the application touches it directly.

5. Architecture Pillars

P1 — Spec-driven shape layer. Generate request parsing, validation, and response/error serialization from provider models (AWS botocore/Smithy service-2.json; GCP Discovery docs; Azure OpenAPI/ARM specs). Shapes become exact *by construction*. Applied incrementally: swap one service's serialization at a time, keep its business logic.

P2 — Lifecycle + Operation engine (the missing piece). A provider-neutral async state machine: per-resource-type states + timed transitions advanced by a tick loop. Create returns a pending resource / **GCP Operation** / AWS resource in `pending`; the tick advances it; describe/waiters/poll observe it. One mechanism serves AWS waiters and GCP/Azure long-running operations.

P3 — Auth-acceptance middleware. Accept (do not reject) SigV4 / GCP bearer / Azure tokens. Additive; existing handlers untouched.

P4 — Conformance harness (the judge). Run real boto3/aws-cli, GCP clients, and the real `hashicorp/aws` & `hashicorp/google` Terraform providers against the sim in CI; emit per-service parity %. Gates every correction.

P5 — Console-as-API + CloudSim reposition. Console already routes through the API, so it inherits lifecycle for free. CloudSim becomes the timing/illustration layer behind P2 (or is demoted), fed the live resource graph (today it only gets create-time counts — a known loose end).

6. Reuse Map (existing code)

- **KEEP (assets):** React console + provider-skin registry; `_SpaceScopedDictProxy` space isolation; `SQLiteStateStore /snapshots`; EC2 LXD/Multipass runtime + bridge + console WS; FirestoreEngine; Terraform export/workflow; packaging/launcher; modular `providers/*`.
- **CORRECT in place (additive):** API shapes/errors per service; lifecycle (wrap create/describe); auth middleware; move S3 bytes → MinIO; make IAM enforcement real where needed.
- **SHIFT method (incremental):** payload exactness via spec-driven serialization; CloudSim reposition.

7. Phased Roadmap

- **Phase 0 — Ground truth (now):** local dev loop + conformance harness; measure AWS S3/EC2/IAM baseline with real clients. Convert the gap matrix into a measured scoreboard.
- **Phase 1 — Vertical slice to "exact":** AWS **S3** first (smallest lifecycle, MinIO data plane, fastest Terraform round-trip), then **EC2** and **IAM**. Gate: real aws-cli/boto3 + real Terraform `apply` then clean `plan` + async console states.
- **Phase 2 — Build the factory:** generalize spec-loader + lifecycle engine + serializers so a new service is "model + transitions + backing", validated by the same harness.
- **Phase 3 — Scale horizontally:** more AWS services; bring **GCP** through the factory; SDK transport
 - console parity ride along.
- **Phase 4 — Azure:** add as the third provider profile through the same factory.
- **Phase 5 — Seamless export:** harden Terraform exporter so the sim graph emits provider-valid HCL that applies unchanged to real cloud; prove via round-trip on Phase-1 services.

8. Acceptance Tests (per service)

1. Real CLI op returns shape-equivalent payload to the real provider (modulo IDs/timestamps).
2. Real SDK (boto3 / Java v2 / Go v2 / GCP client) works with only endpoint override.
3. Real Terraform provider `apply` creates the resource; subsequent `plan` is clean.
4. Lifecycle is observable over time (pending→ready→deleting), waiters/Operations resolve.
5. Console action maps to the same API and reflects the same states.

9. Working Agreement

- Local fast loop: `.venv/bin/python -m uvicorn server:app --port 9000` with a scratch state file.
- Every correction is proven by the harness before moving on.

- Incremental: never break a currently-passing client to chase a new one.